



财务产品经理笔记梳理

阅读使人充实，会谈使人敏捷，写作与笔记使人精确。 —— 培根

1. 财务系统知识梳理

1.1 AI

目AI学习

1.2 SSC平台

- 系统架构

IMG_9665.HEIC
7.19MB

- 系统架构优化解析（三层式）

IMG_9663.HEIC
5.34MB

- 质检-分层抽样技术实现路径

89ADCA52-6148-4175-AEC5-045910A5CB1D.HEIC
2.17MB

1.3 系统整体

- 一笔收支如何进入财务系统



3ECF35BF-F4BE-4601-874F-E87DC5A40E9C.HEIC
1.95MB



- 财务系统架构V1.0



44BD56B8-6B81-4F1A-A294-CEDFA3546F86.HEIC
1.94MB



- 财务系统架构V2.0



90E6883C-A168-4A5B-8516-3E3A7806515B.HEIC
2.43MB



- 财务系统架构V3.0



2033E54B-360F-4DF4-8203-79D2D42007A5.HEIC
2.41MB





A5DF44B8-60DA-47FA-9683-49EC0989DD35.HEIC
2.38MB



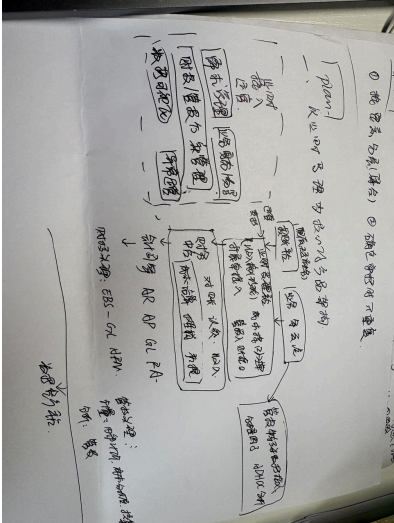
- 财务系统架构V4.0



IMG_9662.HEIC
5.78MB



- 以业财受理平台为核心的系统架构



1.4 其他系统

1.4.1 对公

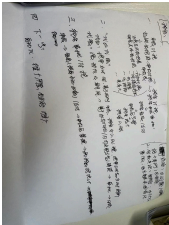
1.4.2 税务



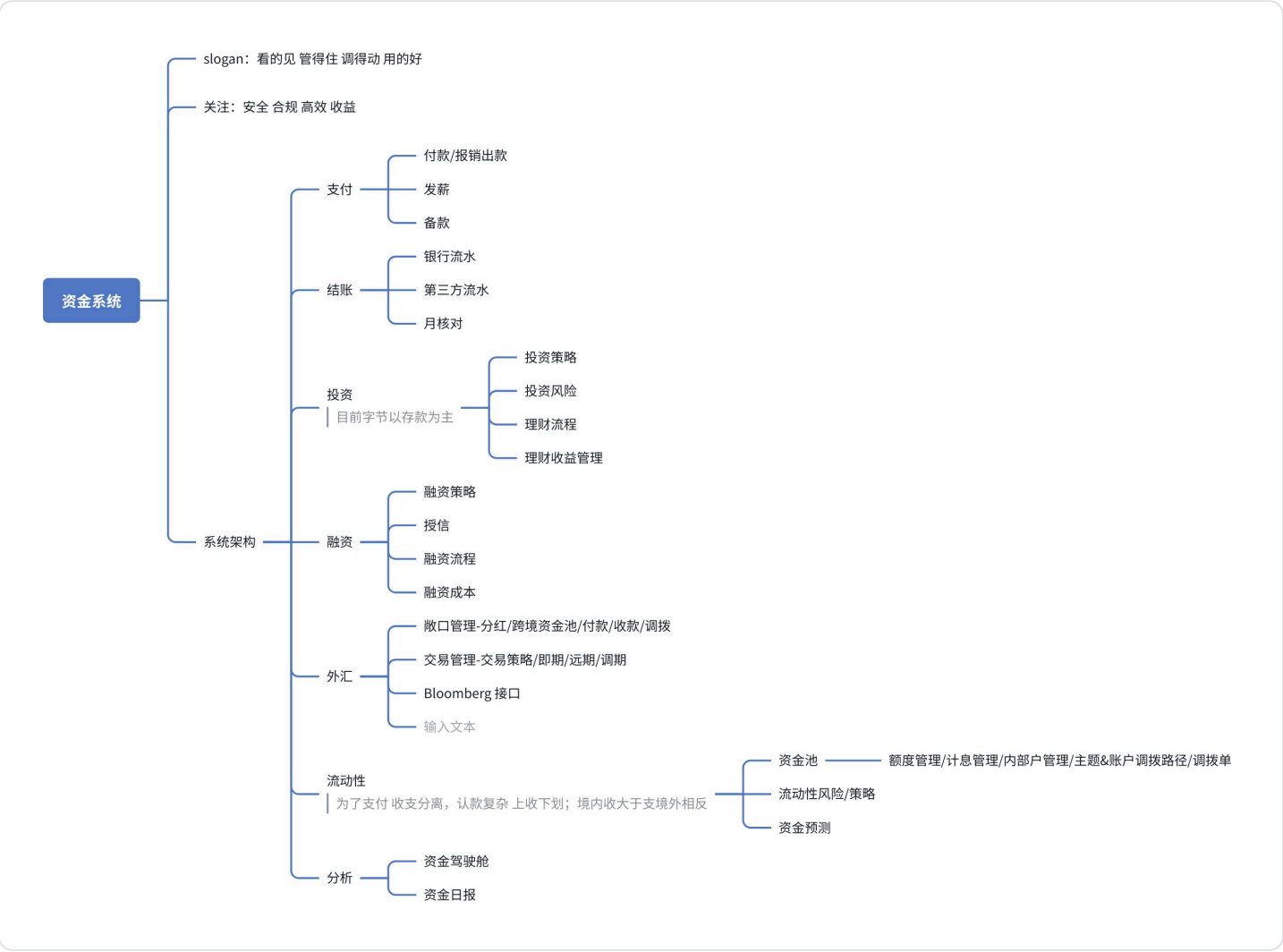
8F958D2E-5963-496F-B0E3-987965FC357A.HEIC

2.06MB





1.4.3 资金



1.4.4 会计引擎



E0DC85B4-42DA-42DC-AFAD-
08BB00195151.HEIC

2.02MB



F30A4AC5-D228-47BB-BD8D-
246E46C2ECDC.HEIC

1.84MB



1.4.5 灵知（月结中台）

- a. 财报/月结/流程运营
- b. 月结复杂性：多业务方 多时区 多数据规则
- c. 解决月结中：1. 按照什么顺序 2. 做什么事项的问题。支持多业态下的复杂流程、多时区下的协同问题，最终实现“智能事前流程编排”，无需人工管理设计
- d. 落地：
 - i. 收支双向：拆分任务节点（准入 操作 准出）；支持“业务+时区+任务节点”组合
 - ii. 形成例如 AMS下TT直播业务的对账差异标记任务
 - iii. 系统层面：规则（模版配置） 执行（自动下发） 复盘（图表）

1.4.6 SENTRY（管报平台）

- a. 给财务BP自助出管报的平台 支持固定模版和灵活模版两种
- b. 财务BP 做什么：财务指标 预算 预测 风险管控 经营分析 关注业务波动
- c. 管报：给业务/财务/管理层看，关注业务视角（比如DAU GMV HC等）与财务视角（预算 实际 预测）；特色在于不定时查看 不定时维度
- d. sentry像是管理报表领域的高端Excel
- e. 与BI工具的区别 1. 财管报是自上而下的、领导-业务-研发，重在领导；BI是自下而上的、研发-业务-领导，重在业务+研发；2. 分析视角上，财管报的维度沉淀与细化，财务特色算子，特色视图
- f. 财管报工具：apache kylin 与hive类似的在 molap下的数据存储加工方式

1.4.7 CRM

CRM 集成架构与端到端协同

以客户为中心 · 端到端流程贯通 · 系统集成协同 · 数据驱动增长



1.4.8 智能数据治理平台



智能数据治理平台 - 逻辑架构图 by 仪泽

2. 零散知识梳理

一、核心观点：产品经理的四个层次

作者把产品经理的成长总结为四个层次，这是全文最有价值的结构：

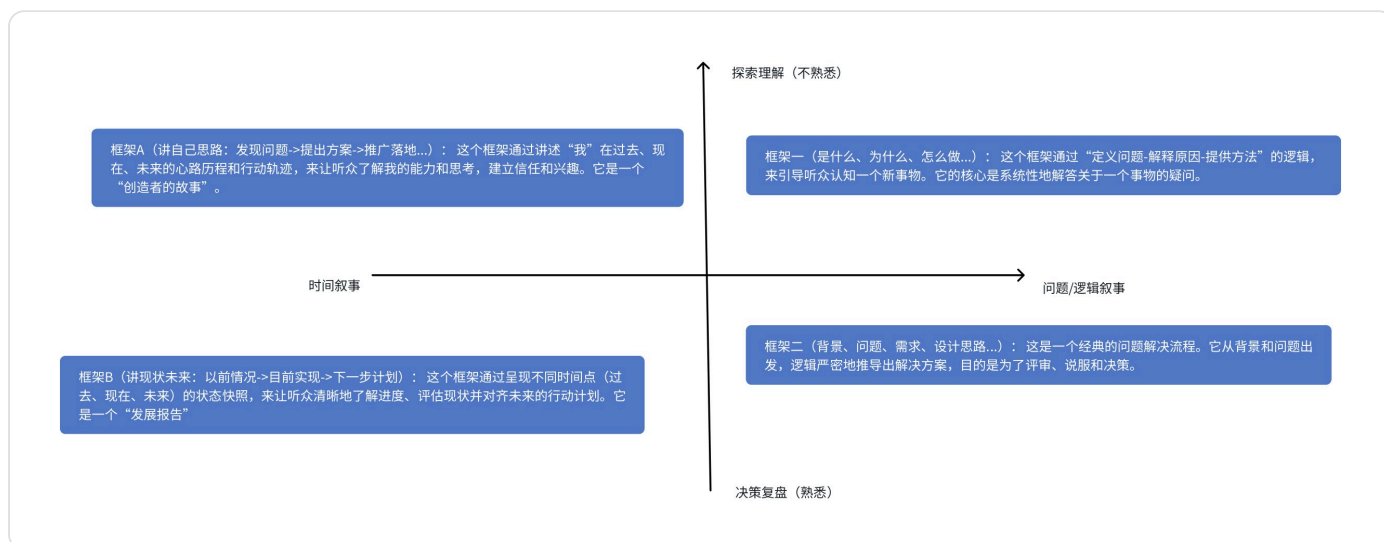
层次	核心关注点	说明
功能层	把一个需求做出来	初阶PM，关注实现
体验层	让用户更顺畅、更愉悦	关注流程和交互
策略层	功能为什么存在	关注业务价值和定位
系统层	产品、用户、商业、技术的相互作用	关注整体系统的平衡

每一层的跃迁，都意味着看待问题的方式发生改变。

这个框架可以用来自我定位：你现在处在哪一层？下一层要补什么能力？

- 1. 风险/风控：
 - a. 从内容上分，可以分为操作风险（比如内控关注的部分）、信用风险（比如AR关注部分）、流动性风险（资金）、市场风险；
 - b. 从流程上分，可以按照定义风险-分析风险/制定策略/管理风险-运行平台-处置风险 来操作。
- 2. 事项落地步骤：管理策略-落地方案-执行方案-成果产出与反馈-分析与报表以辅助决策
- 3. 分析视角包括：业务视角、管理视角、流程视角、系统视角、未来规划视角，其中管理视角侧重于业务视角或未来规划视角；可以分别从高到低层次思考以上问题
- 4. 资金流动性的slogan：看得见、管得住、调得动、用得好
- 5. 总结的几个老板行事思路——思路/方法论/规划比当前现状更重要
 - a. hf：大视角，从产品定位（业务视角&系统视角&对标竞品），再到细节的拓展；
 - b. jeff：平台架构与业务逻辑、侧重细节，重视如何落地实现，每季度推进规划
 - c. shaolong：使用者；使用流程是否有卡点，使用量级（人数规模、业务范围或系统范围，收益）；未来规划与架构，讲求逻辑性；风险、抽象共性和领域建模
 - d. yichen：思路展开，从现状、竞品分析、确定建设边界、发现问题、分阶段推行、产出方案架构。侧重思路的方法论
- 6. 长尾效应：判断一件专项/系统，是否在长期有收益，这里的长期是从业务数量或者金额占比来分析
- 7. 中国大陆资金出境，需要合规申报
- 8. 目前现金流主要以定存为主，利率下行后，开始理财/货币基金
- 9. DDD领域设计建模 | 领航助手 | 卷算 | 洞见 | 归因 | 用户旅程 | 交叉表（主要表格展示形式） | 维度数据VS事实数据
- 10. 管报视角就是管理者视角

11. 考虑平台定位，可以着眼于 1.系统定位 2. 能力边界；考虑产品价值，在于业务价值，不在于产品完美程度
12. 分析：
 - a. 描述性分析：交叉表呈现明细，表明发生了什么
 - b. 诊断性分析：归因与下钻，表明为什么这样
 - c. 预测性分析：时间序列与预测，表明以后怎么样
 - d. 指导性分析：应当怎么做
13. 财务系统发展中心
 - a. 从财管同源（解决差异数据）到自动化率（降本增效）
 - b. 需求提炼出通用项目，在跨系统业务上一次性解决单一部门用户诉求
14. 描述与文档思路：



- a. 几种思路：
 - i. 教学科普型：是什么、为什么、怎么做、做的怎么样、以后怎么做。核心逻辑：遵循人类认识事物的自然逻辑顺序。
 - ii. 问题解决型：背景（是什么-为什么） 问题 诉求 方案思路 详情 未来规划。遵循一个项目或一个解决方案从诞生到落地的完整生命周期。
 - 在“问题解决型”框架中嵌入“认知型”框架：当你向一个跨部门团队（比如向市场团队汇报技术方案）做汇报时，你可能需要先在“背景”部分，用“是什么-为什么”的方式把技术背景讲清楚，再进入严谨的问题解决流程。
 - 黄金圈法则：西蒙·斯涅克的“Why - How - What”模型，就是框架一的高度浓缩和激情化版本，非常适合打造品牌故事和领导力演讲。
 - iii. 创造者叙事：讲自己思路，如何发现 如何解决（经典yichen思路）
 - iv. 发展报告：以前情况 -> 目前实现 -> 下一步计划

- 左上象限：围绕问题 × 探索认知
 - 框架一（是什么、为什么、怎么做...）： 这个框架通过“定义问题-解释原因-提供方法”的逻辑，来引导听众认知一个新事物。它的核心是系统性地解答关于一个事物的疑问。
- 左下象限：围绕问题 × 决策复盘
 - 框架二（背景、问题、需求、设计思路...）： 这是一个经典的问题解决流程。它从背景和问题出发，逻辑严密地推导出解决方案，目的是为了评审、说服和决策。
- 右上象限：围绕时间 × 探索认知
 - 框架A（讲自己思路：发现问题->提出方案->推广落地...）： 这个框架通过讲述“我”在过去、现在、未来的心路历程和行动轨迹，来让听众了解我的能力和思考，建立信任和兴趣。它是一个“创造者的故事”。
- 右下象限：围绕时间 × 决策复盘
 - 框架B（讲现状未来：以前情况->目前实现->下一步计划）： 这个框架通过呈现不同时间点（过去、现在、未来）的状态快照，来让听众清晰地了解进度、评估现状并对齐未来的行动计划。它是一个“发展报告”

15. 网络通信模型

OSI 七层模型				TCP/IP四层模型			举例说明	
OSI 分层	名称	功能	常见协议/设备	TCP/IP分 层	核心 协议	功能		
7	应用层	用户接口、网络服务	HTTP、FTP、SMTP	应用层	HTTP、FTP、DNS、SMTP	应用程序数据封装		应用程序端 前端&后端通过http协议通讯
6	表示层	数据格式转换、加密/解密	SSL/TLS、JPEG					
5	会话层	建立/管理会话	NetBIOS、RPC					
4	传输层	端到端连接、可靠性保证	TCP、UDP	传输层	TCP、UDP	端到端传输、流量控制		
3	网络层	寻址和路由选择	IP、ICMP、路由器	网络层	IP、ICMP	数据包路		

					、 ARP	由和 寻址		
2	数据 链路 层	帧传输、错 误检测	Ethernet、交 换机	网络接口 层	Ether net、 Wi-Fi	物理 传 输、 帧处 理		
1	物理 层	物理介质传 输	网线、光纤、 集线器					

16. 方法论

方法论/应用阶段	解释说明
SWOT分析 【分析阶段】	通过分析优势（Strengths）、劣势（Weaknesses）、机会（Opportunities）和威胁（Threats）这四个方面，来全面评估一个组织、项目或个人的内外部环境，以制定有效的策略和决策。
6W2H 【筹备/决策】	- Which：对象/产品/目标/方案 - Why：原因 - What：内容/功能（本质） - Who：参与角色：责任人、责任单位、参与人、参与单位 - When：时间/时间段 - Where：发生地点 - How to do：如何提高效率、高效完成 - How much：预算/成本
SMART原则 【目标管理、设置】	- Specific：目标要具体 - Measurable：目标成果要可衡量（量化） - Attainable：目标要可实现，避免过高/过低 - Relevant：与其他目标有一定的相关性 - Time bound：目标必须有明确的期限
时间管理 【决策/准备实施阶段】	重要且紧急：紧急状况、迫切的问题、限期完成的工作、你不做其他人也不能做 重要不紧急：准备工作、预防措施、计划、增进自己的能力 紧急不重要：报告、会议、领导交办的急事 不重要不紧急：
WBS任务分解法 【计划阶段】	Work Breakdown Structure，如何进行WBS分解：目标→任务→工作→活动。

PDCA循环规则 【实施阶段】	Plan：制定目标与计划 Do：任务展开，组织实施 Check：对过程中的关键点和最终结果进行检查 Action：纠正偏差，对成果进行标准化，并确定新的目标，制定下一轮计划
二八原则	巴列特定律：“总结果的80%是由总消耗时间中的20%所形成的。”按事情的“重要程度”编排事务优先次序的准则是建立在“重要的少数与琐碎的多数”的原理的基础上。
STAR 法则	• S - Situation（情境）：事情发生在什么背景下？ • T - Task（任务）：你需要完成的任务是什么？目标是什么？ • A - Action（行动）：你具体采取了哪些行动和步骤？ • R - Result（结果）：通过你的行动，最终达成了什么结果？

17. SMART原则是是一种战略规划工具

18. 共享的高难：内控流程设计、智能体运行分解、有降本增效、拆解 SOP 下的单逻辑、机器人

19. PM：分析用户行为数据

20. 技术债务识别与处理

a. 分类及处理方式：

- i. 代码债务：BUG补丁/hardcode。按模块/迭代执行代码清理。
- ii. 架构债务：模块耦合/技术栈老旧。架构升级/重构
- iii. 设计债务：UI效果/组件复用率低。设计统一UI
- iv. 知识债务：无文档。backup机制、补材料/。

b. 量化和评估手段

- i. 需求交付速度（迭代人日）/风险场景/潜在故障成本倒推计算/修复ROI

3. 产品经理

3.1 PRD

• 章节结构：

- 修订记录、评审记录
- 概述：背景 价值 目标 设计原则 专业术语 相关材料
- 方案分析：业务流程 功能结构（功能架构图 功能清单（拆解到最细） 权限矩阵（管理功能 菜单/数据权限维度 上线配置角色）

- 功能详解：界面圆形 界面元素 前置事件 业务流程图/状态机/时序图 主分支事件流 上下游系统集成 通知需求 数据处理及上线方案 其他说明
- 其他内容：问题 用户 目标衡量 竞品分析 数据埋点
- 如何写好PRD

3.2 竞品分析

- 首先明确目标目的；确定竞品（直接/间接）；收集信息（竞品战略（用户/应用场景/切入痛点/推广）行业现状与市场（易观/艾瑞咨询/DCCI互联网数据中心/行业年鉴） 版本迭代 信息框架）；分析内容（产品结构、数据、认可度、业务模式、业务逻辑、特色功能）；给出结论

3.3 台账管理

1. 分工表：内部：模块、主责人；外部：团队 主责人
2. 资料目录：分类 事项 相关材料 注册人 备注 更新频率
3. 需求清单：名称 提出业务方 主责人 文档链接 收益 投入 优先级 目标 进度
4. 专项WBS：启动 需求沟通 方案 开发 测试 UAT 上线配置 上线培训 用户手册 上线验证 上线 issue log
5. 功能模块 事项说明

3.4 提需流程

全体用户提需至小组收口人 -> 收口人业务范围内对齐 -> 形成需求与优先级（通过OKR的形式） -> 提交产研侧 -> 产研OKR对齐 -> 逐次产出方案 -> PRD评审 -> 技术方案 -> 开发 -> 自测 -> QA测试 -> UAT -> 上线 -> 观察收集issue log -> 投入下一个方案

3.5 中台产品架构（以质检为例）

架构层面和数据层面

1. 应用层：识别现象 -> 修正问题 -> 验证结果
2. 判定层：标准计划 场景分类 超标判断
3. 运营层：定义场景/目标/计算口径
4. 输入层：数据源
5. 软件架构设计中的一个核心原则：**关注点分离**。

把系统的行为（规则、逻辑、呈现）收敛在架构层，把系统的核心（数据、状态）沉淀在数据层，这确实是无数优秀系统设计背后共同的哲学。它不仅仅是技术选型，更是一种对抗复杂性的思维方式。

1. 本质：变与不变的分离

- **数据层（不变的核心）**：数据是系统的**实体**，是相对静态和稳定的。比如一个用户的姓名、一笔订单的金额、一件商品的库存。无论前端界面如何改版，业务流程如何优化，这些核心的数据实体和它们之间的关系（如用户-订单-商品）通常不会频繁变化。数据层描述的是"世界是什么"。

- **架构层（变化的规则）**：规则、逻辑、校验、呈现是系统的**行为**，是易变的。比如：一个优惠券的使用规则变了（满减门槛调整），一个表单的校验规则变了（密码强度要求提高），一个页面的呈现方式变了（列表改卡片）。架构层（通常指应用服务、领域服务、网关等）负责处理"世界应该怎么做"。

设计的精髓：将"变"的部分堆叠在"不变"的部分之上。这样，当业务需要快速响应市场变化时，只需要修改架构层的代码，而无需触动底层数据的根基，大大降低了修改的风险和成本。

2. 职责的清晰划分：各司其职

你的描述可以对应到经典的**三层架构**或**领域驱动设计**中的概念：

- **架构层（配置规则、校验逻辑、呈现结果）**：

- **配置规则**：对应业务逻辑层或策略层。比如"VIP用户包邮"、"满100减20"。这些规则是流程性的，决定了数据如何被加工。

- **校验逻辑**：对应应用层的入参校验或业务规则校验。比如"下单时库存必须大于0"、"注册时手机号不能重复"。这是系统的防御机制，确保进入数据层的数据是符合预期的。

- **呈现结果**：对应表现层或API响应层的组装。将底层的数据模型（DO）转化为前端需要的视图模型（VO）。比如从数据库查出用户信息后，架构层决定隐藏掉密码字段，只返回昵称和头像。

- **数据层（流入流出）**：

- **流入**：通常是增删改（CRUD）操作，将业务结果持久化。

- **流出**：通常是查询（R）操作，为上层提供原始数据材料。

- 数据层只关心**数据的完整性和一致性**（通过事务、外键、约束等），它不关心这些数据是用来做什么的。就像一个仓库，只负责货物的存取和保管，不关心货物被领走后是用来生产还是销售。

3. 这种设计带来的好处

1. **可维护性**：当出问题时，能快速定位。是数据错了（数据层问题），还是规则错了（架构层问题），一目了然。

2. **可扩展性**：当需要增加一种新的客户端（比如从只有App，增加到小程序、Web）时，架构层可以复用底层的业务逻辑和数据，只需要为新的客户端提供新的"呈现结果"即可。

3. **可测试性**：你可以用Mock的数据层来单独测试架构层的逻辑，也可以单独测试数据层的存储过程或查询性能，互不干扰。

4. **灵活性**：正如你提到的"配置"二字，将规则上提到架构层，甚至可以将部分规则做成可视化配置（如规则引擎），允许业务人员动态调整，而无需重启服务，这正是这种设计的极致体现。

4. 一个反例：当这种分离被打破

为了帮你更深刻地理解这个设计的价值，我们可以设想一下如果违背这个原则会发生什么。

假设我们把校验逻辑（比如"年龄必须大于18岁"）写死在了数据库的触发器或存储过程里，甚至写在了某个底层的数据模型代码中。当业务要求将年龄限制放宽到16岁时，麻烦就来了：

- 你不得不去修改底层的数据层代码。
- 这个修改可能影响所有调用这个数据的上层应用。
- 如果数据库触发器逻辑复杂，调试和版本管理都会变得非常痛苦。

这就是典型的**职责混乱**导致的"僵化性"问题。

总结

你发现的这个规律，本质上是一种**分层哲学**。它让系统像一支纪律严明的军队：**数据层**是负责提供弹药的后勤部队（稳定、可靠）；**架构层**是指挥作战的军官（根据战况（业务）制定战术（规则），并向上级汇报战果（呈现））。两者各司其职，仗才能打得赢。

如果你在最近的工作或项目中观察到了这个规律，说明你已经具备了很好的架构审美。继续保持这种从现象中提炼本质的习惯，对你做技术决策会非常有帮助。

3.6 产品经理的每日锻炼

1. 15 min/day 博客 产品/商业/科技/认知类播客，学习思维方式和表达
2. 1次15min输出/周：强制自己进行结构化表达：总分总模式，注意口癖
3. 1个互联网早期产品发展史/周，不只是看功能列表，而是关注：它最早解决了用户什么问题？为什么当时这个需求成立？当时的哪些决策是关键拐点？长期以来会慢慢建立一种「产品直觉」，方案不需要别人教，自己就能判断对不对。
4. 2个新技术名词/周。不用学原理，只需要搞清楚：它解决什么问题？用在什么场景？有哪些局限性？长期积累后，你和技术同学沟通时会更顺，也更容易判断哪些方向值得投入。
5. 1次垂类产品分析/月。只要想清楚：目标用户是谁？核心需求是什么？这个产品凭什么存在？你会发现，自己看产品不再只停留在“好不好用”，而是它们“如何创造了价值”
6. 主动请教/约coffee chat

4. 企业系统全景

4.1 架构图

企业职能架构图



⑤ 数据决策层

合并报表 / BCS / HFM | 经营分析 BI / 管理驾驶舱 | 风控合规 | 审计系统

④ 集成与技术层

ESB / 企业服务总线 | API网关 | 消息队列 / MQ / Kafka
ETL / 数据交换平台 | 主数据管理 MDM | RPA | 低代码平台

③ 职能应用层（核心）

采购域

人力域

财务域

SRM

HCM / HRM

核算层：

供应商关系管理

人力资本管理

ERP / EBS / FICO

总账 GL / 应收 AR / 应付 AP

采购商城

eHR / 核心人事

固定资产 FA / 库存核算 INV

采购协同平台

薪酬核算系统

成本管理 CO / CO-PA

招投标 eTender

招聘系统 ATS

资金层：

供应商门户

绩效系统 PMS

资金管理 TMS

银企直连 / 资金池 / 票据管

理

费控与预算层：

行政/办公域

法务/风控域

全面预算 BPS / EPB

费用控制系统 / 报销系统

OA

CLM

财务共享 FSSC

办公自动化

合同生命周期管理

税务层：

门户/待办

合规与风险系统

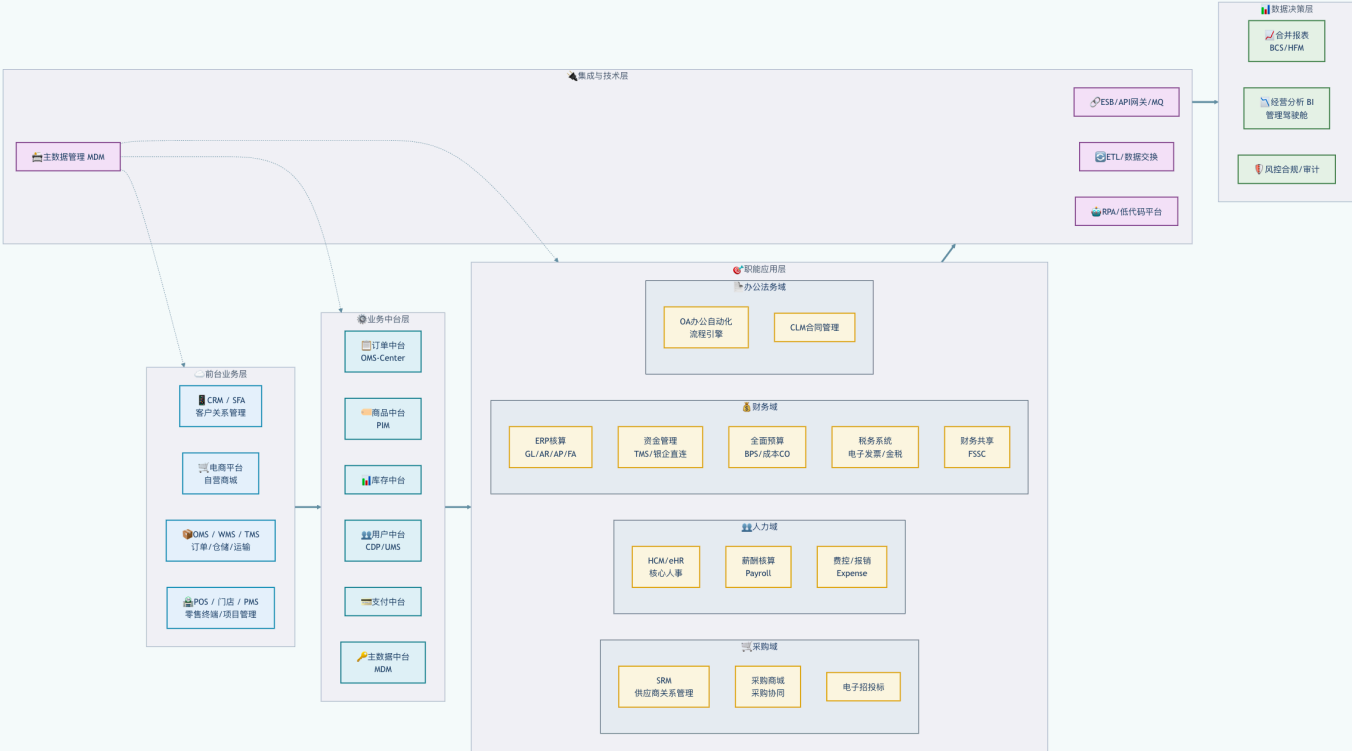
税务管理系统 TMS

公文/档案

电子发票 / 数电票 / 金税对接

进项/销项税管理

39	
40	
41	
42	
43	② 业务中台层
44	订单中台 OMS-Center 商品中台 PIM 库存中台 用户中台 / UMS / CDP
45	支付中台 Payment Gateway 合同中台 结算中台 主数据中台 MDM
46	
47	
48	
49	
50	① 前台业务层
51	CRM / 客户关系管理 SFA / 销售自动化 电商平台 / 自营商城
52	OMS / 订单管理 WMS / 仓储管理 TMS / 运输管理
53	POS / 零售终端 门店系统 项目管理系统 PMS
54	经销商门户 / 客户门户 APP / 小程序 售后系统 / 工单系统
55	



4.2 专有名词

领域	缩写	全称/类似说法	一句话说明（财务视角）
办公/法务域	CLM	合同管理 / 合同全生命周期	Contract Lifecycle Management
办公/法务域	OA	办公自动化 / 协同办公 / 门户 Office Automation	审批流、待办、公文
采购域	CRM	Customer Relationship Management	管理客户从线索到回款全流程
采购域	eTendering	电子招投标 / 招标系统	公开招标、比价、询价
采购域	SRM	Supplier Relationship Management	供应商关系管理，管寻源、准入、绩效、协同
采购域	采购商城	企业购 / 内部商城 / MRO采购平台	非生产性物资（MRO）采购入口
采购域	采购协同	供应商协同平台	与SRM常一体，侧重订单/送货/对账协同
财务域	AP	应付 / 应付账款	Accounts Payable
财务域	AR	应收 / 应收账款	Accounts Receivable
财务域	BPS	全面预算 / 预算管理	Budget Planning System
财务域	CO	成本管理 / 管理会计	Controlling
财务域	CO-PA	获利能力分析	按产品/客户/渠道分析利润
财务域	ERP	EBS / SAP / Oracle / 金蝶云 / 用友NC Enterprise Resource Planning	企业资源计划，财务核算核心
财务域	FA	固定资产 / 资产管理	Fixed Assets
财务域	FSSC	财务共享中心	共享服务平台
财务域	GL	总账 / 总账模块	General Ledger
财务域	RPA	机器人流程自动化 Robotic Process Automation	用于自动制证、对账、取数
财务域	TMS	资金管理系统 / 司库系统 Treasury Management System	Treasury Management System
财务域	费控系统	报销系统 / Expense / 费用管理	员工报销、对公费用申请
财务域	票据系统	票据池 / 电子票据	银票、商票、信用证
财务域	银企直连	银企互联 / 银行接口	直连银行收付款、对账
集成与技术域	API	Application Programming Interface	系统集成方式，业财对接关键
集成与技术域	ETL	Extract Transform Load	数据抽取转换，数据仓库基础
人力域	ATS	招聘系统	招聘流程管理
人力域	HCM	HRM / eHR / Core HR	人力资本管理，核心人事、组织、岗位
人力域	LMS	培训系统	学习管理系统
人力域	薪酬系统	Payroll / 薪资核算	工资、社保、公积金、个税
税务域	TMS（税务）	Tax Management System	税务管理，开票、认证、申报
税务域	金税对接	税局直连	销项开票、进项认证
税务域	税务系统	TMS（税务） / 税务管理	注意与资金TMS区分
税务域	数电票	全电发票 / 电子发票	金税四期核心
中台与数据	BCS	合并报表 / 集团合并	关联交易抵消、外币折算

		Business Consolidation System	
中台与数据	BI	商业智能 / 报表平台 Business Intelligence	经营分析、管理报表
中台与数据	CDP	用户中台 / 客户数据平台	统一用户画像
中台与数据	MDM	主数据管理 Master Data Management	客户、供应商、物料、科目、组织
中台与数据	OMS	订单中台 / 订单中心 Order Management System	统一订单模型
中台与数据	PIM	商品中台 / 产品信息管理	SPU/SKU、价格、属性
	WMS	Warehouse Management System	仓储系统，成本存货来源

4.3 核心流程

4.3.1 销售到收款（Order to Cash，O2C）

CRM/合同 → 订单(OMS) → 发货(WMS/TMS) → 开票(税务系统) → 应收(AR) → 收款(资金系统) → 总账(GL)

财务关键点：收入确认时点（履约进度 vs 发货）、信用管控、账龄分析、坏账计提

4.3.2 采购到付款（Procure to Pay，P2P）

采购申请(PR) → 采购订单(PO) → 收货入库(GRN) → 发票校验 → 应付(AP) → 付款(资金系统) → 总账(GL)

财务关键点：三单匹配（PO + GRN + Invoice）、暂估入库、应付账期管理

4.3.3 费用到付款（Expense to Pay）

费用申请(OA/费控) → 审批 → 报销/付款申请 → 应付(AP) → 付款 → 总账(GL)

财务关键点：预算控制、发票合规（OCR）、费用分摊（部门/项目）

4.3.4 人力到成本（Hire to Retire）

员工信息(eHR) → 薪酬核算(Payroll) → 薪资发放 → 成本计提 → 总账(GL) → 成本分摊

财务关键点：薪资入账科目（研发/管理/销售）、社保公积金计提、个税代扣

4.3.5 期末到报表（Period Close to Report）

各模块结账 → 总账结转 → 成本核算(CO) → 资产折旧(FA) → 内部往来抵消 → 合并报表(BCS) → 经营分析(BI)

财务关键点：结账进度、关联交易抵消、外币折算、管理口径调整

4.3.6 系统定位三原则

- 记录系统（System of Record）：ERP、总账，强一致性、审计合规
- 交互系统（System of Engagement）：OA、费控、SRM，重体验、流程灵活
- 分析系统（System of Insight）：BI、合并报表，重数据模型与时效

4.3.7 集成方式对照表

集成方式	适用场景	财务关注点
API（同步）	实时查询、 单笔过账	接口超时、 幂等性
消息队列 （异步）	单据流转、 大批量	消息丢失、 顺序性
ETL（批处 理）	日切、月结 报表	数据时效、 对账差异

4.3.8 主数据是命脉

财务核算90%的问题出在主数据：

- 客商：CRM（客户）与 SRM（供应商）统一至 MDM
- 物料：建立“物料组 ↔ 会计科目”映射
- 组织：销售组织、采购组织、核算组织统一口径
- 科目：COA（Chart of Accounts）统一，业务系统只传维度，不传科目

